Pandas

Pandas is a powerful open-source Python library that offers versatile data structures and tools for data manipulation and analysis. It's particularly adept at handling structured data, such as tabular data commonly found in CSV files, spreadsheets, SQL databases, and more. Pandas is built on top of NumPy, leveraging its efficient array-based operations while providing additional functionality for data organization and manipulation. Pandas provides easy-to-use data structures and functions for working with structured data.

The key data structures in Pandas are:

1. **Series**: One of the core data structures in Pandas is the Series. A Pandas Series is a one-dimensional labeled array that can hold data of various types, including integers, floats, strings, and Python objects. Each element in a Series is associated with a label or index, allowing for easy and intuitive data access and alignment. Series are similar to one-dimensional NumPy arrays but provide additional functionalities, such as label-based indexing and alignment, making them more flexible and convenient for data analysis tasks.
2. **DataFrame**: A two-dimensional labeled data structure with columns of potentially different types. It can be thought of as a spreadsheet or SQL table, and it is the primary data structure used in Pandas for data manipulation and analysis.

Pandas provides a wide range of functionalities for data manipulation and analysis, including:

Reading and writing data from/to various file formats such as CSV, Excel, SQL databases, and more.

- Handling missing or incomplete data.
- Reshaping and pivoting data.
- Merging, joining, and concatenating datasets.
- Grouping and aggregating data.
- Time series analysis.
- Visualization of data using built-in plotting functions or integration with Matplotlib and Seaborn.

Overall, Pandas is widely used in data science, machine learning, finance, economics, and other fields for data manipulation, cleaning, exploration, and analysis tasks. Its intuitive and powerful API makes it a popular choice among Python developers and data scientists for working with structured data.

Below are some basics of Pandas Series along with examples:

Creating a Pandas Series:

You can create a Pandas Series using pd.Series() constructor by passing a list or array-like object as data.

```
import pandas as pd

# Create a Pandas Series from a list
data = [1, 2, 3, 4, 5]
series = pd.Series(data)
print(series)
```

Accessing Elements:

You can access elements in a Pandas Series using integer indexing or labels.

```
import pandas as pd

# Create a Pandas Series from a list
data = [1, 2, 3, 4, 5]
series = pd.Series(data)
# Accessing elements by integer indexing
print(series[0])  # Output: 1

# Accessing elements by label
series = pd.Series(data, index=['a', 'b', 'c', 'd', 'e'])
print(series['c'])  # Output: 3
```

Operations on Series:

You can perform various operations on Pandas Series, such as arithmetic operations, boolean indexing, and applying functions.

```
import pandas as pd
# Arithmetic operations
series = pd.Series([1, 2, 3, 4, 5])
print(series * 2)

# Boolean indexing
print(series[series > 3])
```

```
# Applying functions
print(series.apply(lambda x: x ** 2))
```

Handling Missing Data:

Pandas Series can handle missing or NaN (Not a Number) values using the NaN keyword from NumPy.

```
import pandas as pd

# Create a Pandas Series from a list
data = [1, 2, 3, 4, 5]
series = pd.Series(data)
print(series)

print(series.shape)
print(series.size)
print(series.dtype)
print(series.mean())
print(series.std())
print(series.min())
print(series.max())
```

Various Functionalities

Here are different functionalities of Pandas Series:

```
import pandas as pd

# Creating a Series from a tuple
data = (1, 2, 3, 4, 5)
s = pd.Series(data)
print("Series from list:")
print(s)
print()

# Creating a Series from a list
data = [1, 2, 3, 4, 5]
s = pd.Series(data)
print("Series from list:")
print(s)
print()

# Creating a Series with custom index
data = {'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}
s = pd.Series(data)
print("Series with custom index:")
```

```
print(s)
print()

# Accessing elements by index
print("Accessing element by index 'b':", s['b'])
print()

# Slicing
print("Slicing from index 'b' to 'd':")
print(s['b':'d'])
print()

# Arithmetic operations
s1 = pd.Series([1, 2, 3, 4, 5])
s2 = pd.Series([6, 7, 8, 9, 10])
print("Addition:")
print(s1 + s2)
print("Subtraction:")
print(s1 - s2)
print("Multiplication:")
print(s1 * s2)
print("Division:")
print(s1 / s2)
print()

# Aggregation functions
print("Mean:", s.mean())
print("Sum:", s.sum())
print("Min:", s.min())
print("Max:", s.max())
print()

# Boolean indexing
print("Boolean indexing for elements greater than 2:")
print(s[s > 2])
print()

# Element-wise functions
print("Applying square function element-wise:")
print(s.apply(lambda x: x ** 2))
print()

# Check if any value is null
print("Is any value null in the Series?", s.isnull().any())
print()

# Check if any value is NaN
print("Is any value NaN in the Series?", pd.isna(s).any())
print()

# Dropping null values
s_with_null = pd.Series([1, 2, None, 4, 5])
print("Series with null values:")
print(s_with_null)
```

```
print("Series after dropping null values:")
print(s_with_null.dropna())
print()

# Replacing null values
print("Series after replacing null values with 0:")
print(s_with_null.fillna(0))
print()

# Sorting
print("Sorted Series by index:")
print(s.sort_index())
print("Sorted Series by values:")
print(s.sort_values())
print()

# Unique values
print("Unique values in Series:", s.unique())
print()

# Value counts
print("Value counts:")
print(s.value_counts())
```

Pandas DataFrame:

A pandas DataFrame is a 2-dimensional labeled data structure with columns of potentially different types. It is similar to a spreadsheet or SQL table, or a dictionary of Series objects. DataFrames are incredibly versatile and are one of the primary data structures used in pandas for data manipulation and analysis. Here's a detailed explanation of pandas DataFrame:

Key Characteristics:

1. **2-Dimensional Structure**: DataFrames are inherently two-dimensional structures, meaning they consist of rows and columns.
2. **Columnar Structure**: Each column in a DataFrame can have a different data type (integer, float, string, etc.). Columns can be thought of as Series objects.
3. **Labeled Axes**: DataFrames have row and column labels that enable easy access to data. Row labels are usually referred to as the index, while column labels are the column names.

DataFrame From Lists/Arrays:

You can create a pandas DataFrame from a list of lists, where each inner list represents a row in the DataFrame. Here's an example:

```python
import pandas as pd

# Creating a list of lists
data_list = [
    [1, 'Alice', 25],
    [2, 'Bob', 30],
    [3, 'Charlie', 35],
    [4, 'David', 40],
    [5, 'Eve', 45]
]

# Creating a DataFrame from the list
df_from_list = pd.DataFrame(data_list, columns=['ID', 'Name', 'Age'])

print(df_from_list)
```

- We can specify column names explicitly using the columns parameter when creating the DataFrame.
- Pandas automatically assigns numeric indices (0, 1, 2, ...) to the rows unless specified otherwise.
- This DataFrame has three columns: 'ID', 'Name', and 'Age'.

DataFrame From Dictionaries:

You can create a pandas DataFrame directly from a dictionary, where the keys are column names and the values are lists containing the column data. Here's an example:

```python
import pandas as pd

# Creating a dictionary
data_dict = {
    'ID': [1, 2, 3, 4, 5],
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [25, 30, 35, 40, 45]
}

# Creating a DataFrame from the dictionary
df_from_dict = pd.DataFrame(data_dict)

print(df_from_dict)
```

DataFrame From CSV, Excel, SQL, or other data sources:

Pandas provides functions like pd.read_csv(), pd.read_excel(), pd.read_sql(), etc., to create a DataFrame from various data sources.

```
import pandas as pd

# Assuming you have a CSV file named 'data.csv' with columns 'col1', 'col2',
'col3'
# Adjust the file name and column names according to your CSV file
file_path = 'data.csv'

# Read the CSV file into a Pandas DataFrame
df = pd.read_csv(file_path)

# Display the first few rows of the DataFrame
print(df.head())
```

Below is an example of how you can read a specific number of rows and select particular columns from a CSV file using Pandas:

```
import pandas as pd

# Assuming you want to read first 10 rows and only columns 'col1' and 'col2'
# Adjust the file name, column names, and nrows according to your
requirement
file_path = 'data.csv'
selected_columns = ['col1', 'col2']
nrows = 10

# Read the specific columns and rows from the CSV file into a Pandas
DataFrame
df = pd.read_csv(file_path, usecols=selected_columns, nrows=nrows)

# Display the DataFrame
print(df)
```

Reading a large CSV file in smaller, manageable chunks is a common practice, especially when dealing with very large datasets. Here's an example of how you can read a CSV file in smaller chunks using Pandas:

```
import pandas as pd

# Assuming you want to read the CSV file in chunks of 1000 rows at a time
# Adjust the file name and chunk size according to your requirement
file_path = 'data.csv'
chunk_size = 1000
```

```
# Create an iterator for reading the CSV file in chunks
chunk_iter = pd.read_csv(file_path, chunksize=chunk_size)

# Iterate over each chunk
for chunk_number, chunk in enumerate(chunk_iter):
    print(f"Chunk {chunk_number + 1}:")
    print(chunk)
```

When using pd.read_csv() with the chunksize parameter, it returns an iterator of DataFrame chunks, each representing a portion of the CSV file. Each chunk is essentially a DataFrame containing a subset of the data from the CSV file, with the number of rows specified by the chunksize parameter.

Therefore, in the loop for chunk_number, chunk in enumerate(chunk_iter):, the variable chunk represents each DataFrame chunk read from the CSV file.

Adding New Row

In Pandas, the DataFrame.insert() method is used to insert a new column into a DataFrame at a specified location. Here's an example demonstrating how to use DataFrame.insert():

```
import pandas as pd

# Create a DataFrame
data = {'A': [1, 2, 3],
        'B': [4, 5, 6]}

df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Insert a new column named 'C' at index 1 with values [10, 20, 30]
df.insert(loc=1, column='C', value=[10, 20, 30])

# Display the DataFrame after insertion
print("\nDataFrame after inserting column 'C':")
print(df)
```

Output:

```
Original DataFrame:
   A  B
0  1  4
1  2  5
```

```
2  3  6

DataFrame after inserting column 'C':
   A   C  B
0  1  10  4
1  2  20  5
2  3  30  6
```

Using pop() to Remove a Column:

Using pop() is useful when you need to remove a column and use its values elsewhere in your code. Below are examples of how to use pop() to remove a column from a DataFrame in Pandas:

```
import pandas as pd

# Create a DataFrame
data = {'A': [1, 2, 3],
        'B': [4, 5, 6],
        'C': [7, 8, 9]}
df = pd.DataFrame(data)

# Remove column 'C' using pop()
column_c = df.pop('C')

# Display the DataFrame after removing column 'C'
print("DataFrame after using pop() to remove column 'C':")
print(df)
```

Using del to Remove a Column:

Using del is more concise and straightforward if you simply want to remove a column from the DataFrame. Below are examples of how to use del to remove a column from a DataFrame in Pandas:

```
import pandas as pd

# Create a DataFrame
data = {'A': [1, 2, 3],
        'B': [4, 5, 6],
        'C': [7, 8, 9]}
df = pd.DataFrame(data)

# Remove column 'C' using del
del df['C']

# Display the DataFrame after removing column 'C'
print("DataFrame after using del to remove column 'C':")
print(df)
```

Removing Missing values Rows and Columns:

The dropna() method in Pandas is used to remove rows or columns with missing values (NaNs). Here's an example of how to use dropna() to remove rows with missing values:

```
import pandas as pd
import numpy as np

# Create a DataFrame with missing values
data = {'A': [1, 2, np.nan, 4],
        'B': [np.nan, 6, 7, 8],
        'C': [9, 10, 11, 12]}
df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Drop rows with missing values
df_cleaned = df.dropna()

# Display the DataFrame after dropping rows with missing values
print("\nDataFrame after dropping rows with missing values:")
print(df_cleaned)
```

Output:

```
Original DataFrame:
     A    B   C
0  1.0  NaN   9
1  2.0  6.0  10
2  NaN  7.0  11
3  4.0  8.0  12

DataFrame after dropping rows with missing values:
     A    B   C
1  2.0  6.0  10
3  4.0  8.0  12
```

You can also use dropna() to remove columns with missing values by specifying axis=1:

```
# Drop columns with missing values
df_cleaned = df.dropna(axis=1)

# Display the DataFrame after dropping columns with missing values
print("\nDataFrame after dropping columns with missing values:")
print(df_cleaned)
```

Output:

```
DataFrame after dropping columns with missing values:
    C
0   9
1  10
2  11
3  12
```

This removes columns containing any missing values, resulting in a DataFrame containing only columns without missing values.

The thresh parameter in the dropna() method allows you to specify a threshold for the number of non-null values required to keep a row or column. Rows or columns with fewer non-null values than the specified threshold will be dropped. Here's an example of how to use dropna() with the thresh parameter:

```
import pandas as pd
import numpy as np

# Create a DataFrame with missing values
data = {'A': [1, np.nan, np.nan, 4],
        'B': [np.nan, 6, 7, np.nan],
        'C': [9, 10, 11, np.nan]}
df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Drop rows with at least 2 non-null values
df_cleaned = df.dropna(thresh=2)

# Display the DataFrame after dropping rows with fewer than 2 non-null
values
print("\nDataFrame after dropping rows with fewer than 2 non-null values:")
print(df_cleaned)
```

Modifying DataFrame inplace:

Setting inplace=True modifies the DataFrame in place, and there's no need to assign the result back to df. This can be convenient for modifying DataFrames directly without creating a new variable. However, be cautious when using inplace=True, especially when working interactively or in scripts, to avoid unintentionally modifying your original data.

```
import pandas as pd
import numpy as np
```

```
# Create a DataFrame with missing values
data = {'A': [1, np.nan, np.nan, 4],
        'B': [np.nan, 6, 7, np.nan],
        'C': [9, 10, 11, np.nan]}
df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Drop rows with missing values in place
df.dropna(inplace=True)

# Display the DataFrame after dropping rows with missing values
print("\nDataFrame after dropping rows with missing values:")
print(df)
```

How Parameter in Dropna:

The how parameter in the dropna() method allows you to specify whether to drop rows or columns based on the presence of any or all missing values (NaNs). The how parameter can take the following values:

- 'any': Drops rows or columns if any NaNs are present in them (default behavior).
- 'all': Drops rows or columns only if all values are NaNs.

Here's an example of how to use the how parameter in dropna():

```
import pandas as pd
import numpy as np

# Create a DataFrame with missing values
data = {'A': [1, np.nan, 3],
        'B': [4, np.nan, np.nan],
        'C': [np.nan, np.nan, np.nan]}
df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Drop rows with all missing values
df_drop_all = df.dropna(how='all')

# Display the DataFrame after dropping rows with all missing values
print("\nDataFrame after dropping rows with all missing values:")
print(df_drop_all)

# Drop columns with any missing values
df_drop_any = df.dropna(axis=1, how='any')
```

```
# Display the DataFrame after dropping columns with any missing values
print("\nDataFrame after dropping columns with any missing values:")
print(df_drop_any)
```

Filling Missing Values:

The fillna() method in Pandas is used to fill missing (NaN) values with a specified value or method. fillna() provides flexibility in handling missing values by allowing you to specify a constant value or use statistical methods like mean, median, or mode for filling missing values based on your data analysis requirements.

Here's an example demonstrating how to use fillna():

```
import pandas as pd
import numpy as np

# Create a DataFrame with missing values
data = {'A': [1, np.nan, 3],
        'B': [np.nan, 5, np.nan],
        'C': [7, 8, np.nan]}
df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Fill missing values with a specific value (e.g., 0)
df_filled = df.fillna(0)

# Display the DataFrame after filling missing values
print("\nDataFrame after filling missing values with 0:")
print(df_filled)

# Fill missing values with the mean of each column
df_filled_mean = df.fillna(df.mean())

# Display the DataFrame after filling missing values with the mean of each
column
print("\nDataFrame after filling missing values with the mean of each
column:")
print(df_filled_mean)
```

Output:

```
Original DataFrame:
     A    B    C
0  1.0  NaN  7.0
1  NaN  5.0  8.0
2  3.0  NaN  NaN
```

```
DataFrame after filling missing values with 0:
     A    B    C
0  1.0  0.0  7.0
1  0.0  5.0  8.0
2  3.0  0.0  0.0

DataFrame after filling missing values with the mean of each column:
     A    B    C
0  1.0  5.0  7.0
1  2.0  5.0  8.0
2  3.0  5.0  7.5
```

When using fillna() with the axis=1 parameter, it fills missing values horizontally along the columns instead of vertically along the rows. Here's an example demonstrating how to use fillna() with axis=1:

```
import pandas as pd
import numpy as np

# Create a DataFrame with missing values
data = {'A': [1, np.nan, 3],
        'B': [np.nan, 5, np.nan],
        'C': [7, 8, np.nan]}
df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Fill missing values horizontally (along columns) using forward fill
(ffill)
df_filled = df.fillna(method='ffill', axis=1)

# Display the DataFrame after filling missing values horizontally
print("\nDataFrame after filling missing values horizontally:")
print(df_filled)
```

Output:

```
Original DataFrame:
     A    B    C
0  1.0  NaN  7.0
1  NaN  5.0  8.0
2  3.0  NaN  NaN

DataFrame after filling missing values horizontally:
     A    B    C
0  1.0  1.0  7.0
1  NaN  5.0  8.0
2  3.0  3.0  3.0
```

We can also use other methods such as backward fill (bfill), interpolation, or specify a constant value when filling missing values horizontally using fillna() with axis=1, depending on your specific requirements and data analysis needs.

Attributes and Methods:

Attributes:

- shape: Returns a tuple representing the dimensionality of the DataFrame (rows, columns).
- index: Returns the row index labels.
- columns: Returns the column names.
- dtypes: Returns the data types of each column.

Methods:

- head(), tail(): View the first or last few rows of the DataFrame.
- info(): Provides a concise summary of the DataFrame including index dtype and column dtypes, non-null values, and memory usage.
- describe(): Generates descriptive statistics for numerical columns.
- loc[], iloc[]: Accessing a group of rows and columns by label(s) or integer position(s).
- drop(): Drop specified labels from rows or columns.
- fillna(), dropna(): Handle missing data by filling or dropping NaN values.
- groupby(), pivot_table(): Perform grouping and aggregation operations.
- merge(), join(): Combine DataFrames through database-style join operations.

Example:

```
import pandas as pd

# Creating DataFrame from dictionary of lists
data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 35],
        'City': ['New York', 'Los Angeles', 'Chicago']}
df = pd.DataFrame(data)
print(df)

# Accessing elements
print(df['Name'])
print(df.iloc[0])

# Adding new column
df['Gender'] = ['Female', 'Male', 'Male']
```

```
print(df)

# Descriptive statistics
print(df.describe())
```

Advantages of Using DataFrame:

- Data alignment and handling missing data.
- Easy manipulation of data (filtering, sorting, grouping).
- Built-in visualization tools for quick data exploration.
- Efficient handling of large datasets.

Pandas DataFrame is a powerful tool for data analysis and manipulation, widely used in data science, machine learning, and statistical analysis. Its flexibility and rich set of functionalities make it an essential component in the Python data ecosystem.